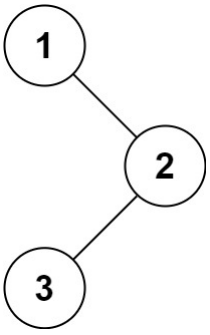
Binary Tree

leetcode

94.二叉树的中序遍历

- 遍历二叉树，返回中序遍历序列。



- 思路：递归遍历。左根右遍历序列。

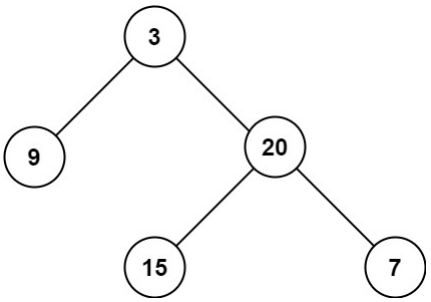
```
class Solution {
public:
    void inorder(TreeNode* root,vector<int>& res){
        // 中序遍历-左根右
        if(!root){ // 终止条件
            return;
        }
        // 左结点
        inorder(root->left,res);
        // 根节点
        res.push_back(root->val);
        // 右结点
        inorder(root->right,res);
    }

    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> res;
        inorder(root,res);
        return res;
    }
};
```

-

95.二叉树的最大深度

- 给定一个二叉树 `root` ，返回其最大深度（根节点到最远叶节点之间的节点数）。




```
输入: root = [3,9,20,null,null,15,7]
输出: 3
```

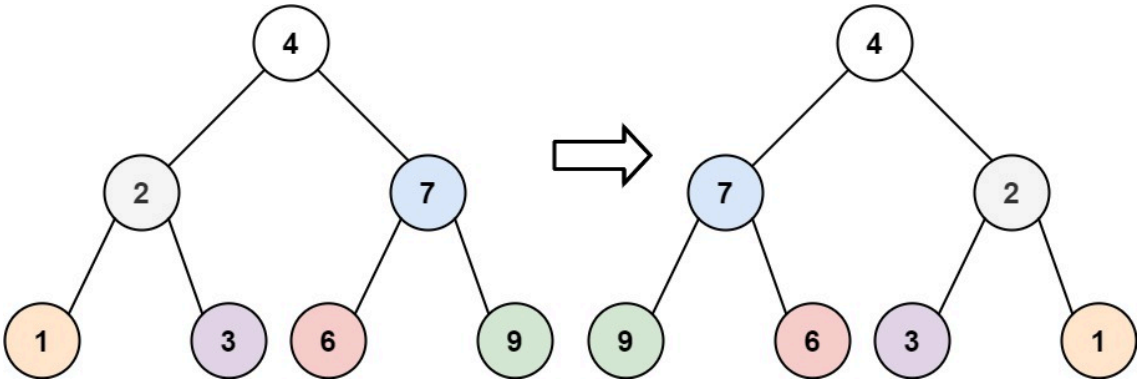
- 思路：递归遍历。最大深度=1+max(左子树最大深度，右子树最大深度)。

```cpp
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if(root == nullptr) {
            return 0;
        } else {
            int left = maxDepth(root->left);
            int right = maxDepth(root->right);
            return std::max(left, right) + 1;
        }
    }
};
```

-

226.翻转二叉树

- 反转二叉树，镜像对称。




```
输入: root = [4,2,7,1,3,6,9]
输出: [4,7,2,9,6,3,1]
```

- 思路：递归遍历。先将左右子树均反转完，然后再交换当前节点的左右子树。

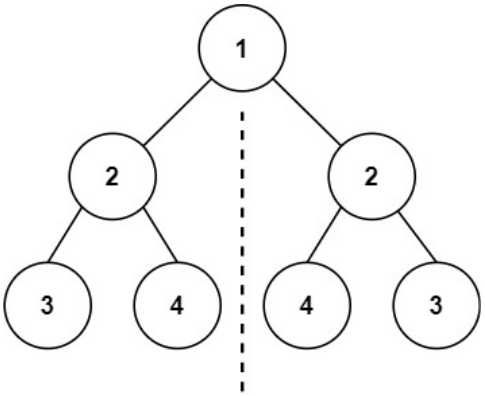
```cpp
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if( root==nullptr ) return nullptr; // 终止条件
        // 反转左子树
        TreeNode *left = invertTree(root->left);
        // 反转右子树
        TreeNode *right = invertTree(root->right);
```

```
        // 交换当前节点的左右子树
        root->left = right;
        root->right = left;
        return root;
    }
};
```

-

101.对称二叉树

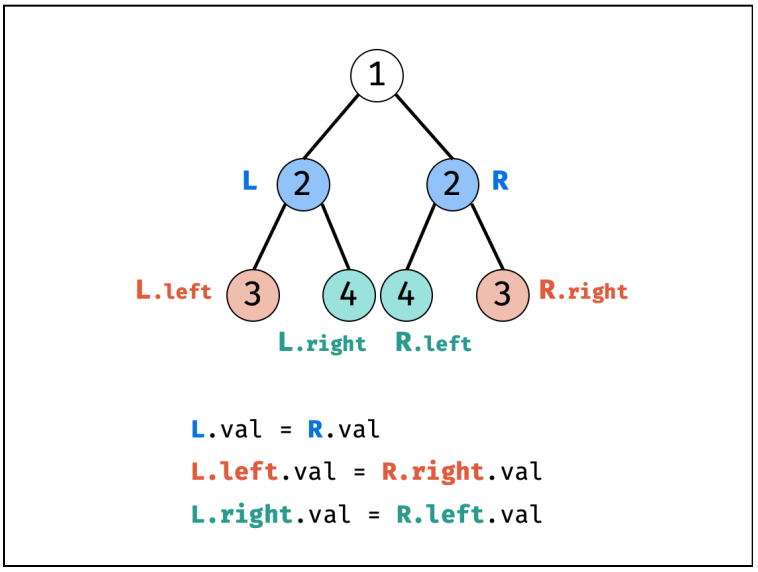
- 检查一棵二叉树是否轴对称。



- 思路：递归遍历。

函数 `recur(L, R)` **：**

- **终止条件：**
  - 当 `L` 和 `R` 同时越过叶节点： 此树从顶至底的节点都对称，因此返回 *true* 。
  - 当 `L` 或 `R` 中只有一个越过叶节点： 此树不对称，因此返回 *false* 。
  - 当节点 `L` 值 $\neq$ 节点 `R` 值： 此树不对称，因此返回 *false* 。
- **递推工作：**
  - 判断两节点 `L.left` 和 `R.right` 是否对称，即 `recur(L.left, R.right)` 。
  - 判断两节点 `L.right` 和 `R.left` 是否对称，即 `recur(L.right, R.left)` 。
- **返回值：** 两对节点都对称时，才是对称树，因此用与逻辑符 `&&` 连接。

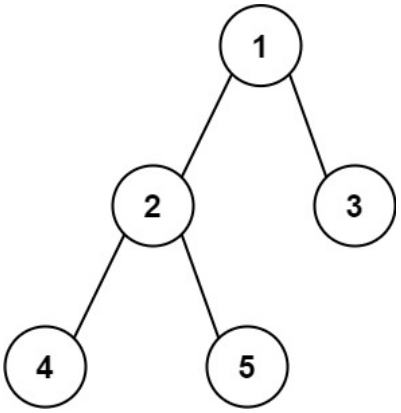


```
class Solution {
public:
    bool isSymmetric(TreeNode* root) {
        if(root==nullptr) return true;
        else return recur(root->left,root->right);
    }
    bool recur(TreeNode* L,TreeNode* R){
        if(L==nullptr&&R==nullptr) return true;
        if(L==nullptr||R==nullptr||L->val!=R->val) return false;
        return recur(L->left,R->right)&&recur(R->left,L->right);
    }
};
```

-

543.二叉树的直径

- 给一棵二叉树，返回树的直径。二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度** 。



输入: root = [1,2,3,4,5]
输出: 3
解释: 3 , 取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

- 思路：左子树最大长度+右子树最大长度

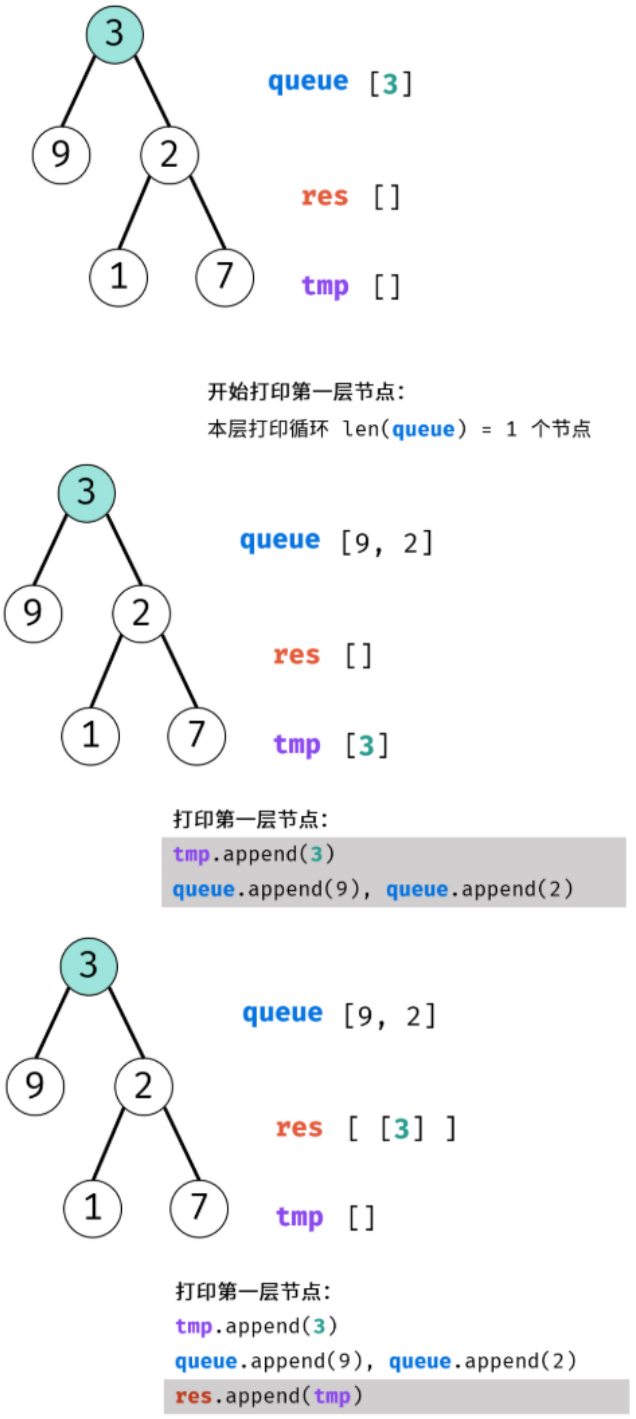
```
class Solution {
    int res = 0; // 存储全局最大值
public:
    int diameterOfBinaryTree(TreeNode* root) {
        res = 0;
        maxDepth(root);
        return res;
    }

    int maxDepth(TreeNode * root){
        if(root==nullptr) return 0;
        int LH = maxDepth(root->left);
        int RH = maxDepth(root->right);
        res = max(res,LH+RH); // 局部最大值，经过当前 root 节点的路径长度=左深度 +右深度
        return 1+std::max(LH,RH); // 最大深度
    }
};
```

-

102.二叉树的层序遍历

- 返回二叉树层序遍历的结果。
- 思路：二叉树的BFS广度优先搜索。借助**队列**来实现。将本层全部节点打印到一行，并将下一层全部节点加入队列，以此类推，即可分为多行打印。



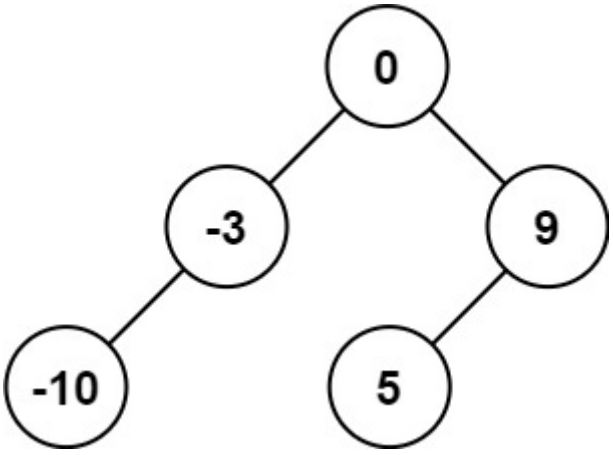

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        vector<vector<int>> res;
        queue<TreeNode*> q;
        if (root != nullptr) q.push(root);
        while(!q.empty()){
```

```
            vector<int> temp; // 用来存储该层的结果
            int levelSize = q.size(); // 当前层的元素个数
            for(int i=0;i<levelSize;i++){
                TreeNode* head = q.front();
                q.pop();
                temp.push_back(head->val);
                // 将该层结点的左右孩子压入队列中
                if( head->left!=nullptr ) q.push(head->left);
                if( head->right!=nullptr ) q.push(head->right);
            }
            res.push_back(temp);
        }
        return res;
    }
};
```

-

108.将有序数组转换为二叉搜索树

- 给定一个按升序排列的数组，转换为平衡二叉树（**平衡二叉树** 是指该树所有节点的左右子树的高度相差不超过 1，左子节点 < 根节点 < 右子节点）。



输入: nums = [-10,-3,0,5,9]
输出: [0,-3,9,-10,null,5]

- 思路:

```

```

-